

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Thaís dos Reis Soares
Matrícula: 825322

Refatoração de Testes e Detecção de Test Smells

Disciplina: Teste de Software

1. Análise de Smells

Ao realizar a análise manual da suíte de testes do projeto, foi possível identificar a presença de diversos *smells*. A seguir, serão apresentados três *test smells* encontrados durante a análise, juntamente com suas respectivas características e impactos na qualidade dos testes.

1.1 Teste Ansioso

O primeiro *smell* identificado foi o de Teste Ansioso. Esse problema acontece quando um único teste verifica métodos e comportamentos diferentes.

Na suíte de testes analisada, havia um teste que verificava a criação e a consulta de um usuário. Assim, o teste passou a possuir responsabilidades distintas, caracterizando a presença desse *test smell*.

Isso é considerado um problema porque o teste deixa de possuir uma única responsabilidade. Caso ele falhe, torna-se mais difícil identificar rapidamente qual funcionalidade realmente apresentou o erro. Além disso, um problema em um método pode acabar mascarando problemas no outro, dificultando a manutenção da suíte de testes.

1.2 Lógica Condicional

Outro *smell* identificado foi a presença de lógica condicional dentro do teste. Esse problema aparece no teste responsável por desativar usuários comuns, que apresenta uma estrutura de repetição e uma estrutura condicional. Isso é considerado um *test smell*, porque testes automatizados devem ser simples, lineares e determinísticos.

Isso é um problema porque a presença de lógica dentro de teste dificulta sua leitura e manutenção, além de tornar o comportamento do teste menos previsível. Outro problema é que o teste pode passar sem validar nada, caso a lista de usuários esteja vazia.

Além disso, esse teste também pode ser classificado como um teste ansioso, já que ele valida dois cenários diferentes, o de usuário comum e o de administrador.

1.3 Teste Frágil

Este *smell* ocorre quando um teste está acoplado à implementação, e não ao comportamento. Ele aparece no teste responsável pela geração do relatório de usuários.

Assim, o problema desse teste é que ele depende que a *string* gerada pelo relatório tenha uma formatação exata, de modo que qualquer alteração na implementação, como uma simples adição de um espaço ou quebra de linha, faça com que ele quebre. Dessa forma, mesmo que o código tenha um comportamento correto, o teste irá falhar.

2. Processo de refatoração

O teste escolhido como o mais crítico foi o responsável por garantir que o sistema não permita a criação de usuários menores de idade.

A seguir, serão apresentadas as imagens do teste antes, figura 1, e depois da refatoração, figura 2.

```
test('deve falhar ao criar usuário menor de idade', () => {  
  // Este teste não falha se a exceção NÃO for lançada.  
  // Ele só passa se o `catch` for executado. Se a lógica de validação  
  // for removida, o teste passa silenciosamente, escondendo um bug.  
  try {  
    userService.createUser('Menor', 'menor@email.com', 17);  
  } catch (e) {  
    expect(e.message).toBe('O usuário deve ser maior de idade.');  }  
});
```

Figura 1 - teste antes da refatoração

```
test('deve falhar ao criar usuário menor de idade', () => {  
  // Arrange  
  const nome = 'Menor';  
  const email = 'menor@email.com';  
  const idade = 17;  
  
  // Act e Assert  
  expect(() => {  
    userService.createUser(nome, email, idade);  
  }).toThrow('O usuário deve ser maior de idade.');});
```

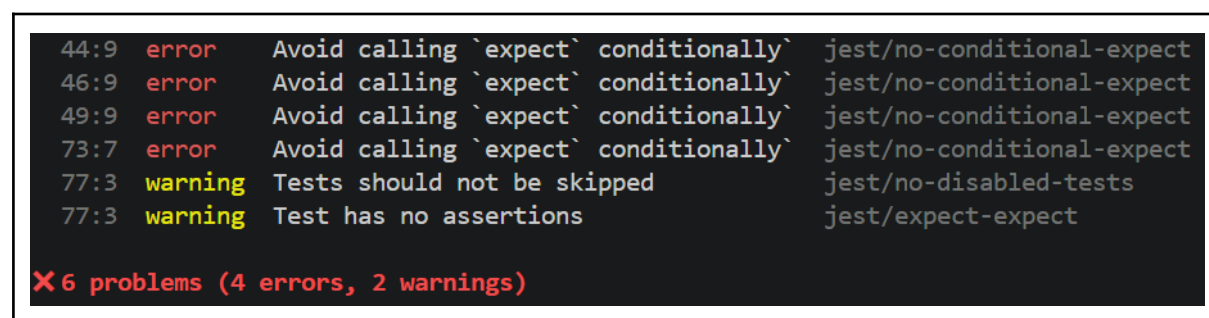
Figura 2 - teste depois da refatoração

O teste original utilizava uma estrutura de controle *try/catch*. Isso é classificado como um *test smell* associado a lógicas condicionais dentro dos códigos de teste. O problema disso é que o *expect* era executado somente se uma exceção fosse lançada dentro do bloco *try*. Se a validação de idade fosse removida da classe principal e a função parasse de lançar erro, o bloco *catch* não seria executado. Com isso, o *expect* também não seria executado e o teste terminaria sem falhas, o que poderia esconder erros no sistema.

Para corrigir esse problema na refatoração, o bloco *try/catch* foi removido, e a responsabilidade da verificação do lançamento de exceção foi passada para o próprio *framework* Jest, através do uso do *matcher toThrow()*. Essa decisão garante que o teste só passará se a exceção exata for disparada, transformando o teste em uma validação determinística, limpa e eficaz.

3. Relatório da Ferramenta

A seguir, a figura três apresenta o resultado da primeira execução da ferramenta ESLint.



```
44:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
46:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
49:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
73:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
77:3  warning  Tests should not be skipped            jest/no-disabled-tests
77:3  warning  Test has no assertions                 jest/expect-expect

✖ 6 problems (4 errors, 2 warnings)
```

Figura 3 - resultado da primeira execução do ESLint

Conforme ilustrado, na primeira execução do ESLint, a ferramenta apontou diversos problemas na suíte de testes original, confirmando a presença dos *test smells* que já haviam sido identificados manualmente. Entre os principais avisos estavam erros como o uso de *expect* dentro de condicionais, testes desabilitados e casos sem nenhuma asserção.

Esse relatório automatizou a detecção dos problemas, evitando a dependência exclusiva da leitura manual do código. Além disso, o resultado do *linter* mostrou exatamente em quais linhas os erros estavam, facilitando o processo de refatoração. E ele detectou os *smells* de forma bem mais rápida, quase que

imediate, em comparação com a detecção realizada por mim de forma manual, o que mostra sua capacidade de tornar esse processo mais eficiente.

E, por fim, a comparação entre o resultado inicial, com múltiplos erros e avisos, e a execução final, sem nenhum erro, evidenciou de forma clara a melhoria da qualidade dos testes após a aplicação das boas práticas. Assim, esse uso da ferramenta reforça como a análise estática contribui para tornar o código de teste mais confiável e sustentável.

4. Conclusão

Este trabalho evidenciou como a combinação entre boas práticas de escrita de testes e o uso de ferramentas de análise estática fortalece a qualidade do *software*.

A refatoração realizada mostrou que testes inicialmente frágeis e cheios de *smells* podem se tornar claros, objetivos e confiáveis quando estruturados pelo padrão *Arrange, Act, Assert* e livres de condicionais ou duplicações.

Já o uso do ESLint com *plugin* do Jest demonstrou ser um grande aliado, pois automatizou a detecção dos problemas e forneceu resultado imediato, permitindo corrigir falhas de forma segura e eficiente.

No resultado final, a suíte de testes não apenas manteve a cobertura, mas passou a oferecer maior segurança e maior legibilidade. Esse processo reforça que investir em testes limpos e em ferramentas de apoio é extremamente importante para a sustentabilidade e evolução contínua dos projetos.